

TYPE COERSION

Type coercion = when JavaScript automatically **converts a value from one type to another** so that an operation can work.

Two Types of Coercion

1. **Implicit Coercion** (JS decides automatically)

JavaScript **guesses** the type based on operators or context.

a) **Number Context**

Operators like -, *, /, <, > → convert values to numbers.

"5" - 2 // 3 (string "5" → number)

"10" * "2" // 20 (both strings → numbers)

true + 1 // 2 (true → 1, false → 0)

JS rules:

true → 1, false → 0

null → 0, undefined → NaN

b) **String Context**

Operator + with **any string** → converts other operand to string.

"5" + 2 // "52"

2 + "5" // "25"

true + "hello" // "truehello"

+ is **special**: if any operand is string, it becomes string concatenation.

c) **Boolean Context (Truthy/Falsy)**

In if statements or logical operators, JS coerces automatically:

Falsy values:

false, 0, -0, 0n, "", null, undefined, NaN

Truthy values: everything else.

```
if ("hello") { console.log("yes"); } // yes
```

```
if (0) { console.log("no"); } // skipped
```

2. Explicit Coercion (we force it using functions)

We can manually convert using JS functions:

```
Number("123") // 123
String(123)    // "123"
Boolean(0)     // false
Boolean("hi")  // true
```

Weird JavaScript Examples

1) == vs ===

```
0 == false // true (coerced)
0 === false // false (no coercion)
"5" == 5    // true
"5" === 5   // false
```

Always prefer === to avoid coercion surprises.

2) null vs undefined

```
null == undefined // true
```

```
null === undefined // false
```

They're treated as equal only with ==, not ===.

3) Adding Arrays / Objects

```
[] + [] // "" (empty string)
```

```
[] + {}    // "[object Object]"  
{ } + []   // 0?    // confusing, depends on parsing
```

Because JS tries to convert objects to primitives.

Objects → string [object Object]

Array → string "" if empty

4) Boolean Coercion in Numbers

```
+true // 1
```

```
+false // 0
```

```
!!"hi" // true
```

```
!!0 // false
```

!! is a common trick to force boolean conversion.

Internal Rules (To Master)

JavaScript uses ToPrimitive conversion:

1. For numbers: try to convert operand → number
2. For strings: try to convert operand → string
3. For boolean context: convert → true/false

Classic Traps

```
[] == ![] // true
```

Step by step:

```
![] → false
```

```
[] == false → [] coerces to "" → "" == false → 0 == 0 → true
```

 Mind-bending, but works once you follow ToPrimitive + numeric conversion rules.

